ABES ENGINEERING COLLEGE, GHAZIABAD
DEPARTMENT OF MECHANICAL ENGINEERING

SOFTWARE REQUIREMENTS SPECIFICATION

AI Driven Digital Twin for Smart Factory Operation

SUBMITTED BY

Anmol Singh
Roll No.: 2400320400010

Degree: Bachelor of Technology (B.Tech)
Academic Year: 2026–2027
Project Status: 3rd Year Project Specification

INSTITUTE DETAILS

ABES Engineering College, Ghaziabad
Department of Mechanical Engineering

Date of Submission: 01/11/2026

Document Version: 1.0
Academic SRS Draft

Document Revision & Approval History

Version	Date	Description / Major Changes	Prepared By	Reviewed By
0.1	30-Oct-2026	Initial SRS draft based on supplied project source code and academic template	Project Team	Project Guide
1.0	To be assigned	Final reviewed baseline after student and supervisor details are confirmed	Project Team	HOD / Committee

Document note: This SRS follows the section structure of the supplied B.Tech template and uses IEEE Std 830-1998 as the stated academic format. Requirements are based on the supplied DigitalTwin-SmartFactory source archive. Items not evidenced in the source code are not represented as implemented features.

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements of the **AI-Driven Digital Twin for Smart Factory Operations**. The document describes the system boundary, operating environment, data interfaces, processing modules, expected outputs, constraints, and verification criteria. It serves as a reference for development, testing, demonstration, and academic evaluation.

1.2 Scope of the System

The system provides a software-based virtual representation of a small smart-factory production line. It simulates three industrial machines, publishes machine telemetry through MQTT, receives telemetry for processing, stores operational records in SQLite, evaluates machine condition using AI and analytical engines, and exposes live factory information through a FastAPI service and browser-based dashboard.

In Scope	Out of Scope / Not Established by Supplied Code
Synthetic machine and production telemetry generation.	Physical deployment of industrial sensors or PLC control.
MQTT publishing/subscribing and telemetry processing.	Direct actuation or safety-critical control of real machinery.
SQLite storage of machine, telemetry, production, alert, health and maintenance records.	Enterprise ERP/MES integration and multi-factory production deployment.
PyTorch anomaly inference, health scoring, operational risk evaluation and maintenance recommendations.	Guaranteed prediction of actual failures or certified industrial safety decisions.
Digital Twin state synchronization and API/dashboard data access.	User registration, login, JWT/RBAC administration; not evidenced in supplied implementation.
OEE calculation and alert generation with cooldown handling.	Commercial cloud SLA or guaranteed 24/7 availability.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
AI / ML	Artificial Intelligence / Machine Learning.
Digital Twin	Software representation of factory machines and their current operational state.
MQTT	Lightweight publish/subscribe messaging protocol used for telemetry transfer.
SQLite	Embedded relational database used for persistent project data.
API	Application Programming Interface; the project exposes HTTP endpoints using FastAPI.
Telemetry	Time-stamped sensor and production measurements from a machine.
OEE	Overall Equipment Effectiveness, calculated from availability, performance and quality.

Term	Definition
Anomaly Score	Model-generated numerical indication of deviation from learned/expected patterns.
SRS	Software Requirements Specification.
FR / NFR	Functional Requirement / Non-Functional Requirement.

1.4 References

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications (format reference supplied in the academic template).
2. Supplied file: *BTech_3rd_Year_SRS_Template.docx* (7-page academic SRS template).
3. Supplied source archive: *DigitalTwin-SmartFactory-main (3).zip* (project implementation reference).
4. Official project dependencies and interfaces as declared in *requirements.txt*: Paho-MQTT, PyTorch, scikit-learn, NumPy, Pandas, FastAPI, Uvicorn, Pydantic, PyYAML, Matplotlib and Plotly.

1.5 Document Overview

Section 2 describes the product context, users, environment and constraints. Section 3 lists functional requirements by module. Section 4 specifies user, hardware, software and communication interfaces. Section 5 defines quality requirements and verification methods. Section 6 presents system analysis models and key data entities. Section 7 provides review and approval fields.

2. Overall Description

2.1 Product Perspective

The product is a modular, software-only smart-factory monitoring prototype. The main processing flow is:

Factory Simulator → MQTT Publisher / Broker → MQTT Subscriber → Processing Pipeline → AI & Analytics → SQLite + Digital Twin → FastAPI / Dashboard / Alerts

The supplied API starts a background simulation publisher and an MQTT subscriber. Incoming machine telemetry is stored and passed through anomaly inference, health evaluation, operational risk evaluation, predictive maintenance diagnosis, OEE calculation, alert evaluation, and Digital Twin synchronization. The SQLite database is also used to hydrate the Digital Twin with the latest stored state during application startup.

2.2 Product Functions

Module	Main Responsibilities
Factory Simulation	Generate synthetic sensor readings and production information for configured machines.
MQTT Communication	Publish and subscribe to telemetry, status, production and alert topics.
Data Persistence	Store machine metadata, telemetry, production summaries, alerts, maintenance events, AI predictions and machine health.
AI Anomaly Detection	Load a trained PyTorch anomaly detector and produce anomaly scores for telemetry.
Health & Risk Analytics	Calculate weighted machine health and operational risk levels using telemetry and AI output.
Predictive Maintenance	Identify a target component and produce maintenance urgency and recommendation.
OEE Analytics	Calculate availability, performance, quality and composite OEE metrics.
Digital Twin	Maintain current virtual state, machine status, sensor values, health, risk, maintenance and OEE.
API & Dashboard	Provide HTTP/WebSocket-oriented application services and browser visualization.
Alert Management	Generate event alerts, apply per-machine/type/severity cooldown and persist/dispatch alerts.

2.3 User Classes and Characteristics

User Class	Technical Proficiency	Responsibilities
Factory Operator / Viewer	Basic to intermediate	View machine status, telemetry, production indicators, warnings and recommendations.
Maintenance Engineer	Intermediate	Review machine health, risk drivers, anomaly indications and maintenance recommendations.
Project Administrator / Developer	High	Configure MQTT and machine parameters, run services, inspect logs, maintain database and execute tests.
Academic Evaluator	Basic to intermediate	Review system behavior, architecture, outputs, test evidence and project documentation.

The supplied implementation does not show a user-account or role-based authentication subsystem. Access control should therefore be treated as a future enhancement unless a separate implementation is provided.

2.4 Operating Environment

Component	Environment / Requirement
Development OS	Windows environment is supported by the project workflow; deployment may use a compatible Python environment.
Runtime	Python 3.10+ is stated in the academic template; the project requirements specify Python packages but do not pin an exact Python version.
Backend	FastAPI with Uvicorn.
Messaging	MQTT broker; local configuration defaults to localhost:1883. TLS and credentials are configurable.
Database	SQLite database file; schema is supplied in database/schema.sql.
AI / Analytics	PyTorch, scikit-learn, NumPy and Pandas.
Client	Modern desktop web browser for the dashboard/API interface.
Configuration	YAML configuration in config/config.yaml; environment variables are supported by MQTT configuration code.

2.5 Design and Implementation Constraints

1. The system depends on an available MQTT broker for live telemetry exchange.
2. Machine IDs and nominal sensor parameters are defined in configuration and must remain consistent across simulator, topics and database records.
3. The database schema uses SQLite and relational foreign-key relationships.
4. The anomaly detector requires its trained model/preprocessor assets to be present and compatible.
5. Synthetic telemetry is intended for demonstration and testing; it must not be treated as certified measurements from physical equipment.
6. The current project is a prototype and is not specified for safety-critical control.

2.6 Assumptions and Dependencies

The project assumes that the Python runtime and declared dependencies are installed, the configuration file is readable, the MQTT broker is reachable, and the SQLite data directory is writable. The dashboard is expected to be accessed through a supported browser. If the broker is unavailable, live message processing may be delayed or unavailable until reconnection succeeds.

3. System Features and Functional Requirements

Each requirement has a unique identifier and a priority. “High” indicates a core requirement for the demonstrated system; “Medium” indicates supporting functionality; “Low” indicates a useful but non-core feature.

3.1 Module 1: Factory Simulation and Telemetry Generation

Req ID	Requirement Description	Input / Validation / Expected Output	Priority
FR-1.1	The system shall initialize the configured virtual factory and its machine instances.	Read factory and machine definitions from config/config.yaml.	High
FR-1.2	The system shall simulate telemetry for the configured machines.	Generate timestamp, machine state, temperature, vibration, current, voltage, RPM, pressure and power values.	High
FR-1.3	The system shall generate production-related measurements.	Include production count, good products, defects, defect rate, cycle time and downtime where available.	High
FR-1.4	The system shall support configurable simulation and publishing intervals.	Use configured interval values; default factory/MQTT interval is approximately one second.	Medium

3.2 Module 2: MQTT Communication and Data Ingestion

Req ID	Requirement Description	Input / Validation / Expected Output	Priority
FR-2.1	The publisher shall connect to the configured MQTT broker.	Broker host, port, credentials, TLS and keepalive settings are configuration inputs.	High
FR-2.2	The publisher shall publish machine telemetry and status messages.	Use configured per-machine topics, e.g. factory/machine1/telemetry and factory/machine1/status.	High
FR-2.3	The subscriber shall subscribe to configured factory topics and receive messages.	Handle machine telemetry, status, production telemetry and alert topic patterns.	High
FR-2.4	The ingestion handler shall route incoming production telemetry separately from machine telemetry.	Production records update production storage and Digital Twin summary.	High
FR-2.5	The system shall attempt reconnection when MQTT connection is unavailable.	Connection retry behavior shall be logged; processing resumes when connectivity returns.	Medium

3.3 Module 3: Database and Data Management

Req ID	Requirement Description	Input / Validation / Expected Output	Priority
FR-3.1	The system shall initialize the SQLite schema and machine metadata.	Create required tables, indexes and views; maintain machine identifiers.	High
FR-3.2	The system shall persist raw machine telemetry.	Store timestamp, machine ID, state and sensor/production values in telemetry records.	High
FR-3.3	The system shall persist production line summaries.	Store total production, good units, defects, defect rate and machine status counts.	High
FR-3.4	The system shall persist machine health and risk history.	Store health score/condition, rule and AI health, anomaly score, risk score/level and primary driver.	High
FR-3.5	The system shall persist alerts and maintenance events/recommendations.	Records shall include time, machine association, type/severity or maintenance details.	Medium
FR-3.6	The system shall retrieve latest telemetry and health records for Digital Twin initialization and API views.	Queries shall support machine-specific and latest-record access.	High

3.4 Module 4: AI Anomaly Detection and Health Evaluation

Req ID	Requirement Description	Input / Validation / Expected Output	Priority
FR-4.1	The system shall load the trained anomaly detector and preprocessing configuration.	Model and preprocessing assets shall be available in the project model directory.	High
FR-4.2	The system shall compute an anomaly score for incoming machine telemetry.	Use the configured feature preprocessing and PyTorch inference pipeline.	High
FR-4.3	The system shall evaluate machine health using configured sensor weights.	Evaluate temperature, vibration, current, RPM, pressure and power factors.	High

Req ID	Requirement Description	Input / Validation / Expected Output	Priority
FR-4.4	The system shall classify machine condition based on calculated health.	Return a health score and condition category such as EXCELLENT, GOOD, FAIR, POOR or CRITICAL, as supported by the evaluator.	High

3.5 Module 5: Operational Risk and Predictive Maintenance

Req ID	Requirement Description	Input / Validation / Expected Output	Priority
FR-5.1	The system shall calculate a multi-factor operational risk score.	Consider health deficit, AI anomaly, physical stress, defect rate and downtime.	High
FR-5.2	The system shall classify operational risk.	Return LOW, MEDIUM, HIGH or CRITICAL according to configured thresholds and state overrides.	High
FR-5.3	The system shall identify the primary risk driver.	Return the strongest contributing factor such as vibration, temperature, anomaly, health, quality or downtime.	Medium
FR-5.4	The system shall generate predictive maintenance recommendations.	Determine urgency, target component and recommended action from machine state, health, risk and available trends.	High
FR-5.5	The system shall record maintenance recommendations/events where configured.	Persist generated maintenance information for later review.	Medium

3.6 Module 6: Production Analytics and OEE

Req ID	Requirement Description	Input / Validation / Expected Output	Priority
FR-6.1	The system shall calculate machine availability.	Use planned operating time and downtime.	High
FR-6.2	The system shall calculate machine performance.	Use ideal cycle time, total production and operating time.	High
FR-6.3	The system shall calculate product quality.	Use good product count divided by total product count where total is non-zero.	High
FR-6.4	The system shall calculate composite OEE.	OEE = Availability × Performance × Quality; report percentage values.	High
FR-6.5	The system shall provide production-line summary metrics.	Summarize production, good units, defects, defect rate and machine status counts.	Medium

3.7 Module 7: Digital Twin, Dashboard and API

Req ID	Requirement Description	Input / Validation / Expected Output	Priority
FR-7.1	The Digital Twin shall maintain one virtual representation for each configured machine.	Track online state, machine state, latest telemetry, health, anomaly, risk, maintenance and OEE.	High
FR-7.2	The Digital Twin shall update when processed telemetry is received.	Synchronize latest machine and production state with processing outputs.	High
FR-7.3	The application shall expose API endpoints through FastAPI.	Provide machine, factory, health, risk, alert and/or telemetry data as implemented by the API routes.	High
FR-7.4	The application shall provide browser-accessible monitoring visualization.	Display machine status and operational/analytics information supported by the frontend.	High
FR-7.5	The application should support live client updates where WebSocket routes are enabled.	Deliver changing state to connected dashboard clients and handle disconnects.	Medium

3.8 Module 8: Alert Management

Req ID	Requirement Description	Input / Validation / Expected Output	Priority
FR-8.1	The system shall evaluate telemetry and analytics for alert conditions.	Consider failure/warning state, sensor thresholds, anomaly, health degradation, risk and maintenance indications.	High
FR-8.2	The system shall classify alerts by type and severity.	Support INFO, WARNING and CRITICAL severity categories and defined industrial alert types.	High
FR-8.3	The system shall apply an alert cooldown to reduce repeated notifications.	Default cooldown in configuration is 10 seconds; suppression is tracked per machine/type/severity.	Medium
FR-8.4	The system shall persist alert records and dispatch supported alert messages.	Store timestamp, machine ID, alert type, severity, message and acknowledgement state.	High

4. External Interface Requirements

4.1 User Interfaces

The browser interface shall present the virtual factory overview and machine-level monitoring information. The visual presentation should make machine state, health condition, operational risk, sensor telemetry, production metrics, alerts and maintenance recommendations distinguishable. Error or unavailable-data states should be communicated clearly. Exact screen dimensions and accessibility conformance are not specified in the supplied source and should be confirmed during UI review.

4.2 Hardware Interfaces

The supplied implementation is simulation-based and does not require physical industrial hardware. Standard development and user peripherals (computer, display, keyboard and mouse) are sufficient. Physical sensors, PLCs, ESP32 boards, GPIO, I²C, SPI or serial interfaces are not established as implemented interfaces in the supplied source archive.

4.3 Software Interfaces

Interface	Specification
Python application	Core runtime for simulator, processing modules, API and tests.
MQTT broker	Paho-MQTT client; local default broker host localhost and port 1883; credentials/TLS configurable.
SQLite	Relational persistence using the supplied schema and database manager.
AI runtime	PyTorch model inference with preprocessing assets; scikit-learn and numerical libraries are declared dependencies.
Web API	FastAPI and Uvicorn; API documentation is available through the framework's OpenAPI/Swagger interface when enabled.

4.4 Communication Interfaces

MQTT is used for publish/subscribe telemetry transfer. Configured topic families include per-machine telemetry and status, production telemetry, and factory alerts. The local default broker endpoint is localhost:1883. MQTT TLS and authentication settings are configurable. The FastAPI service provides HTTP-based client access; WebSocket support is present in the API module for connected live clients. Payloads are represented as structured Python dictionaries/JSON-compatible data.

5. Non-Functional Requirements

ID / Attribute	Requirement / Target	Verification Method
NFR-5.1 Performance	The system should process incoming telemetry continuously at the configured simulation rate (default approximately one update per second per publisher cycle). Dashboard/API response targets must be measured rather than assumed.	Run sustained simulation and record message processing latency, API response time and CPU/memory usage.
NFR-5.2 Reliability	The application should handle temporary MQTT connection loss without crashing and should log connection/processing errors.	Disconnect/reconnect broker during a controlled test; inspect logs and recovery.
NFR-5.3 Data Integrity	Machine telemetry and health records must retain machine identifiers and timestamps; database foreign-key relationships must remain valid.	Run database integrity checks and automated database tests.
NFR-5.4 Availability	Availability depends on the host, broker and network. No production uptime SLA is claimed by the supplied project.	Monitor service process and broker during the academic demonstration.
NFR-5.5 Security	Broker credentials and deployment secrets should be supplied through environment variables or protected configuration. TLS should be enabled when using a secured remote broker.	Review configuration and environment handling; verify TLS/auth settings for remote deployment.
NFR-5.6 Maintainability	Modules should remain separated by responsibility: simulator, MQTT, database, AI, analytics, alerts, Digital Twin and API.	Code review and module-level test execution.
NFR-5.7 Portability	The project should run in a compatible Python environment with declared dependencies installed; browser dashboard should be accessible using a modern browser.	Install in a clean environment and test supported browsers.
NFR-5.8 Testability	Core components shall be verifiable using the supplied pytest test suite.	Execute pytest -v and retain test output.

Note: The supplied academic template contains example targets such as page load below 2 seconds, API response below 350 ms, 50 concurrent users and 98% uptime. These values are not evidenced as measured results for this project and are therefore not claimed as achieved requirements in this draft. They may be adopted as targets after performance testing.

6. System Design and Analysis Models

6.1 Context-Level Data Flow (DFD Level 0)

External entities: Factory Simulation Engine, MQTT Broker, Operator / Maintenance Viewer and Project Administrator. The central system receives machine and production telemetry, processes it, stores records, and provides monitoring information and alerts to users.

External Entity	Data Sent to System	Data Received from System
Factory Simulator	Synthetic sensor values, machine state and production values.	Simulation configuration / control settings.
MQTT Broker	Published telemetry and status messages delivered to subscriber.	Telemetry, status, production and alert publications.
Operator / Maintenance Viewer	API or dashboard requests.	Machine condition, live telemetry, production metrics, risk and recommendations.
Project Administrator	Configuration and service operations.	Logs, system status and test/diagnostic information.

6.2 Level 1 Processing Flow

Step	Process	Output
1	Generate synthetic machine telemetry.	Time-stamped machine/production payload.
2	Publish through MQTT and receive through subscriber.	Incoming telemetry message.
3	Persist raw telemetry in SQLite.	Stored telemetry row.
4	Run AI anomaly inference and health evaluation.	Anomaly score and health condition.

Step	Process	Output
5	Evaluate operational risk and maintenance.	Risk score/level, driver and maintenance recommendation.
6	Calculate OEE and evaluate alerts.	OEE metrics and alert candidates.
7	Update Digital Twin and expose through API/dashboard.	Current virtual factory view and monitoring output.

6.3 Use Case Summary

Actor	Use Cases
Operator / Viewer	Open dashboard; view factory status; inspect machine telemetry; review health, risk, production and alerts.
Maintenance Engineer	Inspect machine degradation; review risk driver; read maintenance urgency and recommendation; review maintenance history.
Administrator / Developer	Configure machine/MQTT settings; start and stop application services; inspect database/logs; run tests.
Academic Evaluator	Review requirements, architecture, system outputs, test evidence and project report.

6.4 Entity-Relationship Model: Main Data Entities

Entity / Table	Key Attributes	Relationship / Purpose
machines	machine_id (unique), name, type, nominal sensor values	Master metadata for configured machines.
telemetry	id, timestamp, machine_id, machine_state, sensor and production values	Many telemetry records belong to one machine.
production	id, timestamp, factory_name, factory_status, production totals and machine counts	Time-series factory-level production summary.
machine_health	id, timestamp, machine_id, health score/condition, anomaly score, risk score/level	Many health evaluations belong to one machine.
ai_predictions	id, timestamp, machine_id, anomaly score, health score, risk level, recommendation	AI/health prediction history associated with a machine.
alerts	id, timestamp, machine_id, type, severity, message, acknowledged	Machine-associated or factory-level alert events.
maintenance_events	id, timestamp, machine_id, event type, description, downtime, technician	Maintenance history associated with a machine.

The supplied SQLite schema defines foreign-key relationships from telemetry, machine_health and ai_predictions to machines. The schema also defines indexes for time-series lookup and views for telemetry with AI information and latest machine overview.

6.5 Sequence Summary: Incoming Machine Telemetry

1. Simulator creates a telemetry payload and publisher sends it to the machine telemetry topic.
2. MQTT subscriber receives the payload and invokes the registered processing handler.
3. The handler stores raw telemetry in SQLite.
4. AI inference returns an anomaly score; health and risk engines calculate machine condition and operational risk.
5. Predictive maintenance and OEE engines generate their respective outputs.
6. Alert manager evaluates alert conditions, applies cooldown and persists/dispatches alerts.
7. Digital Twin updates the machine's in-memory state; API/dashboard clients can read the latest state.

7. Academic Review & Sign-Off

Project Guide / Coordinator	Internal Examiner	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Finalization checklist

- ■ Institute and department details completed
- ■ Student names and roll numbers completed
- ■ Project guide details completed
- ■ Project-specific diagrams/screenshots reviewed and inserted if required by department
- ■ Functional requirements verified against final submitted code
- ■ Guide/examiner approval obtained

Appendix A: Project Configuration Snapshot

The supplied configuration identifies the virtual factory as **Smart Factory Virtual Line 1** and defines three machines:

Machine ID	Machine Name	Type	Nominal Temperature	Nominal Vibration	Nominal RPM
M01	CNC Milling Machine	machining	65.0 °C	2.5 mm/s	1800
M02	Hydraulic Stamping Press	forming	70.0 °C	3.2 mm/s	1200
M03	Robotic Welding Station	assembly	75.0 °C	1.8 mm/s	950

The configuration also defines nominal current, voltage, pressure, power, cycle time, sensor noise, warning/failure thresholds, health weights, risk thresholds and a 10-second alert cooldown.

Appendix B: Suggested Verification Commands

Purpose	Command / Location
Run integrated pipeline	python main_pipeline.py
Start API and dashboard service	uvicorn api.main:app --reload --port 8000
Open API documentation	http://localhost:8000/docs
Run automated tests	pytest -v
Database schema	database/schema.sql
Main configuration	config/config.yaml

These commands and file locations are taken from the supplied project structure and source archive. The API URL is a local development address, not a public deployment URL.